



PY#1 – Sistemas Operativos

Profesor: Christian Campos

Estudiantes:

Julián Lizano Monge - 2024188887

Instituto Tecnológico de Costa Rica

Fecha de entrega: 6/10/2026

1. Introducción

Este documento describe el diseño del sistema operativo simulado: cómo se organizó en paquetes, qué responsabilidad tiene cada uno y cómo se comunican. El sistema simula la gestión de procesos, memoria y disco a partir de programas en ensamblador (.asm).

El diseño sigue los principios de Stallings (7 estados, cambio de contexto, tabla de memoria y tabla de archivos).

2. Diagrama de paquetes



El sistema se organiza en capas: presentación (gui), lógica del sistema operativo (logica), modelo del hardware (modelo) y configuración (config).

3. Descripción de los paquetes

Paquete	Responsabilidad	Clases principales
App	Punto de entrada. Crea y muestra la ventana principal.	Main
gui	Interfaz gráfica. Muestra procesos, memoria, disco y pantalla, y recibe las acciones del usuario (cargar, ejecutar, paso a paso, configurar, estadísticas).	VentanaPrincipal, PanelProcesos, PanelMemoria, PanelDisco, PanelPantalla, VentanaConfiguracion, VentanaEstadisticas, Paleta
logica	Núcleo del S.O.: carga de programas, ensamblado, planificación, despacho, ejecución de instrucciones, interrupciones y asignación de memoria.	GestorProcesos, Ensamblador, Despachador, EjecutorCPU, Interrupciones, ListaProcesos, ListaDeTrabajos, ParticionadorDinamico, Planificador
logica.planificacion	Algoritmos de planificación intercambiables mediante una interfaz.	EstrategiaPlanificacion, PlanificadorFCFS
modelo	Representación de los componentes del hardware y las estructuras de datos.	Memoria, CPU, Disco, BCP, Instruccion, EstadoProceso
config	Lectura, validación y guardado de config.txt, con corrección automática de valores inválidos.	ConfiguracionExterna

4. Dependencias entre paquetes

- App depende solo de gui.
- gui depende de logica (para operar el sistema), de modelo (para mostrar el estado) y de config (para leer y guardar la configuración).
- logica depende de modelo y delega la política de planificación a logica.planificacion.
- modelo no depende de ningún otro paquete, lo que lo hace la base del sistema.

5. Decisiones de diseño

- **Separación en capas:** la interfaz no contiene reglas del S.O.; solo invoca al GestorProcesos. Así se puede cambiar la interfaz sin tocar la lógica.
- **Patrón Strategy en la planificación:** Planificador usa una EstrategiaPlanificacion. Hoy se usa FCFS, pero se pueden agregar otras (Round Robin, SJF) sin modificar el resto.
- **Memoria dividida en kernel y usuario:** el kernel guarda los BCP y el espacio de usuario guarda los procesos, gestionado con particionado dinámico (ParticionadorDinamico).
- **Disco con tres zonas:** índice de archivos, área de swap y área de archivos. El swap permite suspender procesos (READY_SUSPEND, BLOCKED_SUSPEND) cuando no hay espacio en memoria.
- **Cola de trabajos:** si un programa no cabe en memoria, queda en ListaDeTrabajos hasta que haya hueco.
- **Interrupciones:** Interrupciones atiende teclado, pantalla y operaciones de archivos, e indica si el proceso sigue corriendo, se bloquea o termina.
- **Comunicación por callbacks:** el GestorProcesos notifica a la interfaz (salida en pantalla, solicitud de teclado, operaciones de archivo) sin depender de las clases de gui.
- **Configuración externa validada:** config.txt permite cambiar tamaños de memoria, disco, kernel y swap, y se corrige solo si hay valores inválidos.
- **DX como String:** el registro DX almacena el nombre del archivo como cadena de texto para INT 21H (crear, abrir, leer, escribir, eliminar), siguiendo la indicación del profesor.

6. Flujo general de ejecución

1. El usuario carga uno o varios .asm.
2. Ensamblador valida y traduce el archivo a instrucciones.
3. GestorProcesos reserva una partición y un BCP en memoria; si no hay espacio, el proceso va a la lista de trabajos o al swap.
4. Planificador elige el siguiente proceso y Despachador lo carga en la CPU.

5. EjecutorCPU ejecuta las instrucciones; las interrupciones pueden bloquear o terminar el proceso.
6. Al terminar, se libera la memoria y se registra un BCPTerminado para las estadísticas.